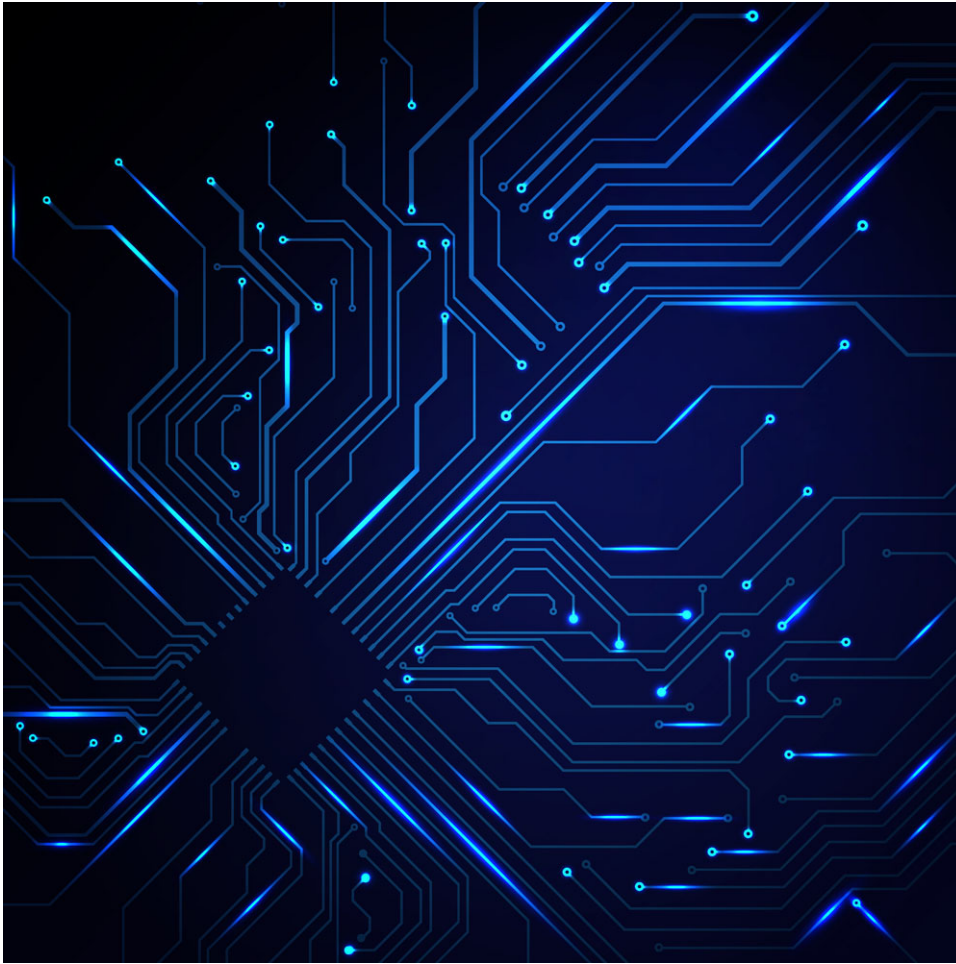




LAB7

COMBINATIONAL LOGIC

Name : 黃翊宏 Student ID : 01357101

Note

1. All files must be converted to PDF format before submission.
2. **Submission deadline: Next Tuesday at 1:00 PM**
3. Each person submits one assignment.

※ Please use Win + Shift + S for screenshots. Do not take photos of the screen.



Lab 7-1 :

Main Code Screenshot

```
design > V ssd.sv
1  module ssd(
2      input logic [3:0] a,
3      output logic [6:0] seg
4  );
5
6      assign seg[0] = ~(
7          | (a[1] & a[2])
8          | (a[1] & ~a[3])
9          | (a[3] & ~a[0])
10         | (~a[0] & ~a[2])
11         | (a[0] & a[2] & ~a[3])
12         | (a[3] & ~a[1] & ~a[2])
13     );
14
15     assign seg[1] = ~(
16         | (~a[0] & ~a[2])
17         | (~a[2] & ~a[3])
18         | (a[0] & a[1] & ~a[3])
19         | (a[0] & a[3] & ~a[1])
20         | (~a[0] & ~a[1] & ~a[3])
21     );
22
23     assign seg[2] = ~(
24         | (a[0] & ~a[1])
25         | (a[0] & ~a[3])
26         | (a[2] & ~a[3])
27         | (a[3] & ~a[2])
28         | (~a[1] & ~a[3])
29     );
30
31     assign seg[3] = ~(
32         | (a[3] & ~a[1])
33         | (a[0] & a[1] & ~a[2])
34         | (a[0] & a[2] & ~a[1])
35         | (a[0] & a[3] & ~a[2])
36         | (a[1] & a[2] & ~a[0])
37         | (a[2] & a[3] & ~a[0])
38         | (a[1] & ~a[0] & ~a[3])
39         | (a[1] & ~a[2] & ~a[3])
40         | (~a[0] & ~a[1] & ~a[2])
41         | (~a[0] & ~a[2] & ~a[3])
42     );
43
44     assign seg[4] = ~(
45         | (a[1] & a[3])
46         | (a[2] & a[3])
47         | (a[1] & ~a[0])
48         | (~a[0] & ~a[2])
49     );
50
51     assign seg[5] = ~(
52         | (a[1] & a[3])
53         | (a[2] & ~a[0])
54         | (a[3] & ~a[2])
55         | (~a[0] & ~a[1])
56         | (a[2] & ~a[1] & ~a[3])
57     );
58
59     assign seg[6] = ~(
60         | (a[0] & a[3])
61         | (a[1] & a[3])
62         | (a[1] & ~a[0])
63         | (a[1] & ~a[2])
64         | (a[3] & ~a[2])
65         | (a[0] & a[2] & ~a[1])
66         | (a[2] & ~a[0] & ~a[3])
67         | (a[2] & ~a[1] & ~a[3])
68     );
69
70 endmodule
71
```

Testbench Screenshot

```
simulation > tb > V tb_ssd.sv
1  module tb_ssd;
2
3      logic [3:0] a;
4      logic [6:0] seg;
5
6      ssd u_ssd(
7          .a(a),
8          .seg(seg)
9      );
10
11     initial
12     begin
13         a=0;
14         #10 a=1;
15         #10 a=2;
16         #10 a=3;
17         #10 a=4;
18         #10 a=5;
19         #10 a=6;
20         #10 a=7;
21         #10 a=8;
22         #10 a=9;
23         #10 a=10;
24         #10 a=11;
25         #10 a=12;
26         #10 a=13;
27         #10 a=14;
28         #10 a=15;
29         #10 $stop;
30     end
31
32     initial
33     begin
34         $monitor($time, " a=%d, g=%b, f=%b, e=%b, d=%b, c=%b, b=%b, a=%b", a, seg[6], seg[5], seg[4], seg[3], seg[2], seg[1], seg[0]);
35     end
36
37 endmodule
```

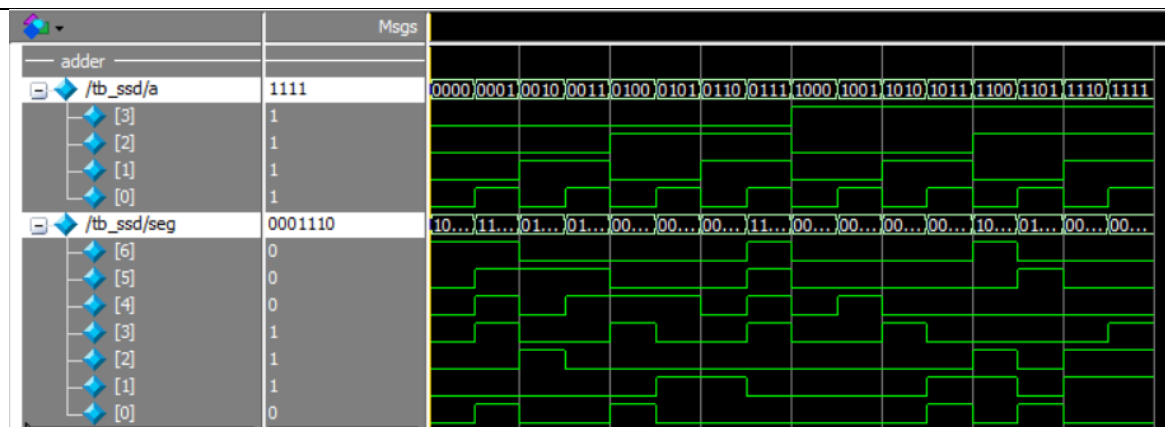
Mcu.sv Code Screenshot

```

6  module mcu(
7
8      //////////// CLOCK ////////////
9      input CLK,
10
11     //////////// 7-segment decoder ////////////
12     output [6:0] HEX0,
13     output [6:0] HEX1,
14     output [6:0] HEX2,
15     output [6:0] HEX3,
16
17     //////////// LED ////////////
18     output [9:0] LED,
19
20     //////////// SWITCH ////////////
21     input [9:0] SW, //上1下0
22
23     //////////// BUTTON ////////////
24     input [2:0] BTN
25
26 );
27 //*****
28
29 // add module here
30 > /*adder_8bit adder1( ...
35 );
36
37 |
38 ssd ssd1(
39     .a    (SW[3:0]),
40     .seg  (HEX0)
41 );
42
43
44 > /*fourSAS fourSAS1( ...
51 );*/

```

Waveform and Result Screenshot



```
# ** Error: (vish-4014) No objects found matching '/testbench/adder1/b'.
# Executing ONERROR command at macro ./wave.do line 14
# ** Error: (vish-4014) No objects found matching '/testbench/adder1/s'.
# Executing ONERROR command at macro ./wave.do line 15
# ** Error: (vish-4014) No objects found matching '/testbench/b[6]'.
# Executing ONERROR command at macro ./wave.do line 16
# ** Error: (vish-4014) No objects found matching '/testbench/b[5]'.
# Executing ONERROR command at macro ./wave.do line 17
#
#       0 a= 0, g=1, f=0, e=0, d=0, c=0, b=0, a=0
#
#      10 a= 1, g=1, f=1, e=1, d=1, c=0, b=0, a=1
#
#      20 a= 2, g=0, f=1, e=0, d=0, c=1, b=0, a=0
#
#      30 a= 3, g=0, f=1, e=1, d=0, c=0, b=0, a=0
#
#      40 a= 4, g=0, f=0, e=1, d=1, c=0, b=0, a=1
#
#      50 a= 5, g=0, f=0, e=1, d=0, c=0, b=1, a=0
#
#      60 a= 6, g=0, f=0, e=0, d=0, c=0, b=1, a=0
#
#      70 a= 7, g=1, f=1, e=1, d=1, c=0, b=0, a=0
#
#      80 a= 8, g=0, f=0, e=0, d=0, c=0, b=0, a=0
#
#      90 a= 9, g=0, f=0, e=1, d=0, c=0, b=0, a=0
#
#     100 a=10, g=0, f=0, e=0, d=1, c=0, b=0, a=0
#
#     110 a=11, g=0, f=0, e=0, d=0, c=0, b=1, a=1
#
#     120 a=12, g=1, f=0, e=0, d=0, c=1, b=1, a=0
#
#     130 a=13, g=0, f=1, e=0, d=0, c=0, b=0, a=1
#
#     140 a=14, g=0, f=0, e=0, d=0, c=1, b=1, a=0
#
#     150 a=15, g=0, f=0, e=0, d=1, c=1, b=1, a=0
# Break in Module tb_ssd at ../tb/tb_ssd.sv line 29
# Simulation Breakpoint: Break in Module tb_ssd at ../tb/tb_ssd.sv line 29
# MACRO ./sim.do PAUSED at line 7
```



Lab 7-2 :

Main Code Screenshot

fourAS.sv:

```
design > V fourAS.sv
1  module fourAS(
2      input [3:0] a,
3      input [3:0] b,
4      input m,
5      output logic c,
6      output logic [3:0] s,
7      output logic v
8  );
9
10 logic C1, C2, C3;
11 logic X0, X1, X2, X3;
12
13 xor XOR0 (X0, b[0], m);
14 xor XOR1 (X1, b[1], m);
15 xor XOR2 (X2, b[2], m);
16 xor XOR3 (X3, b[3], m);
17
18
19 fulla f0 (
20     .A (a[0]),
21     .B (X0),
22     .Cin (m),
23     .S (s[0]),
24     .Cout(C1)
25 );
26
27 fulla f1 (
28     .A (a[1]),
29     .B (X1),
30     .Cin (C1),
31     .S (s[1]),
32     .Cout(C2)
33 );
34
35 fulla f2 (
36     .A (a[2]),
37     .B (X2),
38     .Cin (C2),
39     .S (s[2]),
40     .Cout(C3)
41 );
42
43 fulla f3 (
44     .A (a[3]),
45     .B (X3),
46     .Cin (C3),
47     .S (s[3]),
48     .Cout(c)
49 );
50
51 xor XOR4 (v, C3, c);
52
53 endmodule
```

fulla.sv:

```
design > V fulla.sv
1  module fulla(
2      input logic A,
3      input logic B,
4      input logic Cin,
5      output logic S,
6      output logic Cout
7  );
8      assign S = A ^ B ^ Cin;
9      assign Cout = (A & B) | (A & Cin) | (B & Cin);
10 endmodule
11
```

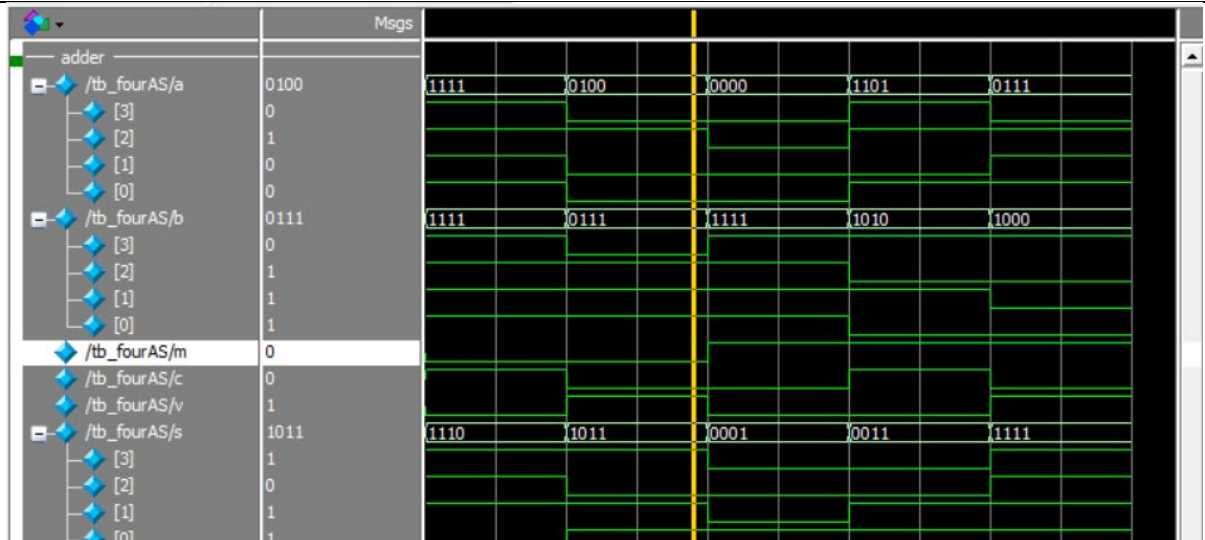
Testbench Screenshot

```

simulation > tb > V tb_fourAS.sv
1  module tb_fourAS;
2
3      logic [3:0]a;
4      logic [3:0]b;
5      logic m, c, v;
6      logic [3:0]s;
7
8      fourAS u_fourAS(
9          .a(a),
10         .b(b),
11         .m(m),
12         .c(c),
13         .s(s),
14         .v(v)
15     );
16
17     initial
18     begin
19         a= 15; b = 15; m = 0;
20         #10 a = 4; b = 7; m = 0;
21         #10 a = 0; b = 15; m = 1;
22         #10 a = 13; b = 10; m = 1;
23         #10 a = 7; b = 8; m = 1;
24         #10 $stop;
25     end
26
27     initial
28     begin
29         $monitor($time, " a=%d, b=%d, m=%b, s=%d, c=%b, v=%b", a, b, m, s, c, v);
30     end
31 endmodule
32
33

```

Waveform and Result Screenshot



Library x sim x

Transcript

```
# Loading work.fourAS
# Loading work.fulla
# .main_pane.structure.interior.cs.body.struct .main_pane.wave.interior.cs.body.pw.wf .main_pane.c
# ** Error: (vish-4014) No objects found matching '/testbench/adder1/a'.
# Executing ONERROR command at macro ./wave.do line 13
# ** Error: (vish-4014) No objects found matching '/testbench/adder1/b'.
# Executing ONERROR command at macro ./wave.do line 14
# ** Error: (vish-4014) No objects found matching '/testbench/adder1/s'.
# Executing ONERROR command at macro ./wave.do line 15
# ** Error: (vish-4014) No objects found matching '/testbench/b[6]'.
# Executing ONERROR command at macro ./wave.do line 16
# ** Error: (vish-4014) No objects found matching '/testbench/b[5]'.
# Executing ONERROR command at macro ./wave.do line 17
#           0 a=15, b=15, m=0, s=14, c=1, v=0
#           10 a= 4, b= 7, m=0, s=11, c=0, v=1
#           20 a= 0, b=15, m=1, s= 1, c=0, v=0
#           30 a=13, b=10, m=1, s= 3, c=1, v=0
#           40 a= 7, b= 8, m=1, s=15, c=0, v=1
# Break in Module tb_fourAS at ../tb/tb_fourAS.sv line 24
# Simulation Breakpoint: Break in Module tb_fourAS at ../tb/tb_fourAS.sv line 24
# MACRO ./sim.do PAUSED at line 7
add wave -position insertpoint sim:/tb_fourAS/*

VSIM(paused)>
```



Lab 7-3 :

Main Code Screenshot

```
design > V fourSAS.v
1  module fourSAS(
2      input [3:0] a,
3      input [3:0] b,
4      input m,
5      output logic [3:0] s,
6      output logic [6:0] seg,
7      output logic c,
8      output logic v
9  );
10
11  logic C1, C2, C3;
12  logic X0, X1, X2, X3;
13
14  xor XOR0 (X0, b[0], m);
15  xor XOR1 (X1, b[1], m);
16  xor XOR2 (X2, b[2], m);
17  xor XOR3 (X3, b[3], m);
18
19  fulla f0 (
20      .A (a[0]),
21      .B (X0),
22      .Cin (m),
23      .S (s[0]),
24      .Cout(C1)
25  );
26
27  fulla f1 (
28      .A (a[1]),
29      .B (X1),
30      .Cin (C1),
31      .S (s[1]),
32      .Cout(C2)
33  );
34
35
36  fulla f2 (
37      .A (a[2]),
38      .B (X2),
39      .Cin (C2),
40      .S (s[2]),
41      .Cout(C3)
42  );
43
44  fulla f3 (
45      .A (a[3]),
46      .B (X3),
47      .Cin (C3),
48      .S (s[3]),
49      .Cout(c)
50  );
51
52  ssd ssd1 (
53      .a (s),
54      .seg (seg)
55  );
56
57  xor XOR4 (v, C3, c);
58
59  endmodule
60
```

The fulla is same as the Lab 7-2.

The ssd is Lab 7-1.

Testbench Screenshot


```

simulation > tb > V tb_fourSAS.sv
1  module tb_fourSAS;
2
3      logic [3:0]a;
4      logic [3:0]b;
5      logic m;
6      logic c, v;
7      logic [3:0] s;
8      logic [6:0] seg;
9
10     fourSAS u_fourSAS(
11         .a(a),
12         .b(b),
13         .m(m),
14         .c(c),
15         .s(s),
16         .v(v),
17         .seg(seg)
18     );
19
20     initial
21     begin
22         a = 15; b = 15; m = 0;
23         #10 a = 4; b = 7; m = 0;
24         #10 a = 0; b = 15; m = 1;
25         #10 a = 13; b = 10; m = 1;
26         #10 a = 7; b = 8; m = 1;
27         #10 $stop;
28     end
29
30     initial
31     begin
32         $monitor($time, " a=%d, b=%d, m=%b, seg=%b, c=%b, v=%b, s=%b", a, b, m, seg, c, v, s);
33     end
34 endmodule

```

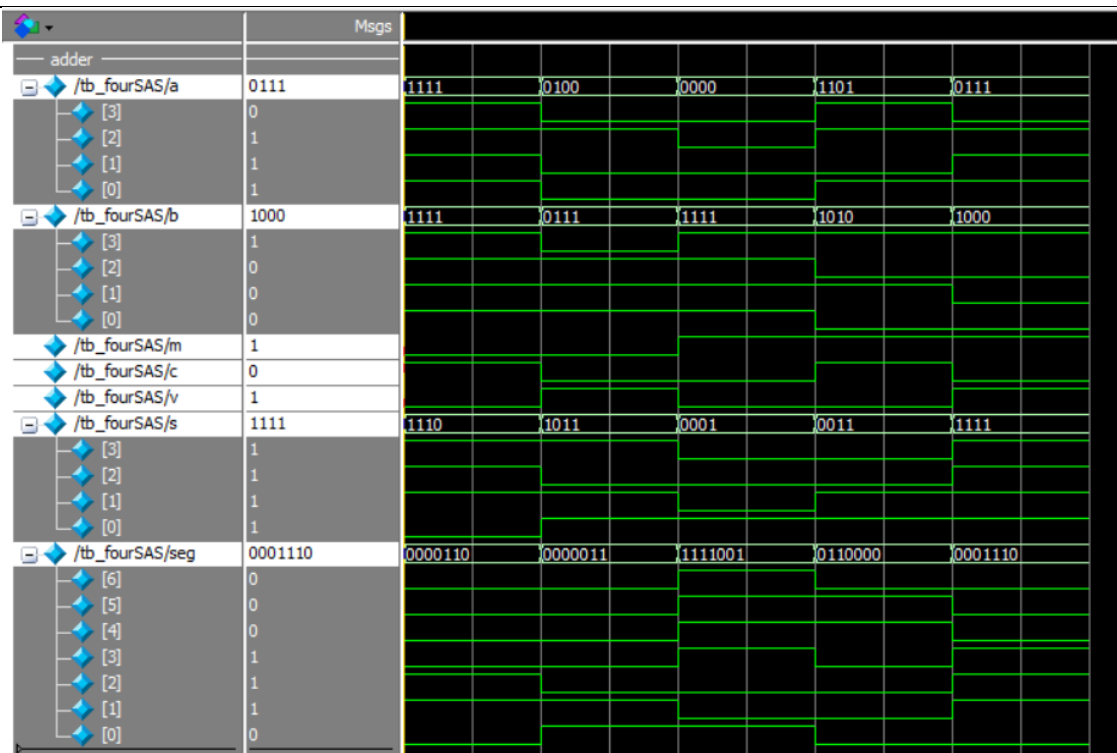
Mcu.sv Code Screenshot

```

29
30 // add module here
31 > /*add8bit adder1( ...
35 );
36
37
38 > /*ssd ssd1( ...
41 );*/
42
43
44 fourSAS fourSAS1(
45     .a(SW[3:0]),
46     .b(SW[7:4]),
47     .m(SW[8]),
48     .seg(HEX0),
49     .c(LED[0]),
50     .v(LED[1])
51 );
52
53
54
55 //*****//
56
57 endmodule

```

Waveform and Result Screenshot



```

Transcript
# Loading work.fulla
# Loading work.ssd
# .main_pane.structure.interior.cs.body.struct .main_pane.wave.interior.cs.body.pw.wf .main_pane.objects.interior.cs.body.tree
# ** Error: (vish-4014) No objects found matching '/testbench/adder1/a'.
# Executing ONERROR command at macro ./wave.do line 13
# ** Error: (vish-4014) No objects found matching '/testbench/adder1/b'.
# Executing ONERROR command at macro ./wave.do line 14
# ** Error: (vish-4014) No objects found matching '/testbench/adder1/s'.
# Executing ONERROR command at macro ./wave.do line 15
# ** Error: (vish-4014) No objects found matching '/testbench/b[6]'.
# Executing ONERROR command at macro ./wave.do line 16
# ** Error: (vish-4014) No objects found matching '/testbench/b[5]'.
# Executing ONERROR command at macro ./wave.do line 17
#
# 0 a=15, b=15, m=0, seg=0000110, c=1, v=0, s=1110
# 10 a= 4, b= 7, m=0, seg=0000011, c=0, v=1, s=1011
# 20 a= 0, b=15, m=1, seg=1111001, c=0, v=0, s=0001
# 30 a=13, b=10, m=1, seg=0110000, c=1, v=0, s=0011
# 40 a= 7, b= 8, m=1, seg=0001110, c=0, v=1, s=1111
# Break in Module tb_fourSAS at ../tb/tb_fourSAS.sv line 27
# Simulation Breakpoint: Break in Module tb_fourSAS at ../tb/tb_fourSAS.sv line 27
# MACRO ./sim.do PAUSED at line 7
add wave -position insertpoint sim:/tb_fourSAS/*

VSIM(paused)>

```



Lab Summary and Reflection:

Summary:

In this week's experiment, we employed Quartus and the Altera DE0 development board to design and simulate fundamental combinational logic circuits using SystemVerilog. First, we implemented a seven-segment display (SSD) decoder that translates a 4-bit binary input into the corresponding segment activation pattern to render hexadecimal digits on a common-anode SSD. Next, we developed a 4-bit adder/subtractor module capable of performing addition or subtraction based on a mode control signal, complete with overflow detection via a 'v' flag. Finally, these two modules were integrated so that arithmetic results could be simultaneously computed and displayed: binary operands entered via the DE0's slide switches are processed by the adder/subtractor, and the resulting sum or difference is fed through the SSD decoder for real-time visualization.

Reflection:

Through this hands-on exercise, I deepened my understanding of mapping abstract Boolean expressions to physical hardware interfaces and the nuances of SystemVerilog's assign statements for combinational logic. Working with ModelSim testbenches highlighted the importance of thorough stimulus verification and waveform inspection to catch edge-case behaviors, particularly in subtraction operations where two's-complement logic can introduce subtle overflow conditions. Downloading the synthesized design to the DE0 board reinforced practical skills in pin-assignment constraints and timing analysis under real-world conditions. Moving forward, I aim to optimize my code for resource utilization and explore parameterized module designs to enhance reusability across similar combinational tasks.